MANCHESTER
1824
The University of Manchester

Group-Agent Reinforcement Learning for Algorithmic Trading

A dissertation submitted to The University of Manchester for the degree of

Master of Science in Machine Learning

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

14210396

Department of Computer Science

Contents

Word count: 2834

Abstract

This is abstract text.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

Acknowledgments

Acknowledgments go here.

1 Introduction

1.1 Background

Financial markets are venues in which financial securities are traded or invested in by individual investors, institutions, and governments. Common tradable instruments include equities, bonds, foreign exchange, commodities, derivatives, and digital assets. Trading involves the purchase or sale of these instruments to generate returns, manage risk, or modify existing exposures. Among these, the stock market is the most widely recognised and is actively traded by the public. A share of stock signifies an ownership interest in a company and a proportional claim on its assets and profits [1].

Stock trading involves determining which assets to hold, the direction and magnitude of each position, and the timing of position changes. These decisions are challenging due to the noisy nature of financial returns, evolving statistical relationships, and erosion of profitable signals by transaction costs. The Efficient Market Hypothesis (EMH) posits that asset prices quickly reflect all publicly available and private information, thereby limiting the persistence of exploitable patterns in historical data [2]. While market efficiency does not preclude the possibility of prediction, it sets a rigorous standard that a model must demonstrate out-of-sample economic value rather than merely fitting historical prices.

Trading approaches can be discretionary or systematic. Discretionary strategies are substantially dependent on human judgement, whereas systematic strategies express decisions through reproducible rules. Quantitative trading uses mathematical, statistical and computational methods to construct such rules. Algorithmic trading refers to automated determination and execution of trading decisions. Although these concepts overlap, they are not hierarchical. A strategy can be quantitatively researched and executed through algorithms, while an execution algorithm may automate orders without generating the initial investment signal.

Decision Making / Execution Method	Discretionary	Systematic
Manual	Human judgment with manual order entry	Systematic/Quant signal plus manual ordering
Algorithmic	Discretionary signal plus algo execution	Fully automated quantitative trading

Table 1. Trading Strategy Taxonomy Matrix

In systematic trading, models are differentiated by how they transform all available information into investment decisions. Traditional statistical forecasting estimates future values based on explicit assumptions about time-series structure. Supervised machine learning establishes functional mappings from explanatory variables to designated prediction targets, whereas deep learning models learn temporal representations from ordered sequential data. In contrast, deep reinforcement learning treats trading as a sequential decision-making process in which actions directly influence future portfolio states and cumulative rewards. These methodological differences motivate two central questions in this research, specifically regarding whether greater modelling capacity produces superior out-of-sample trading performance, and whether reinforcement learning agents can benefit from cross-environment knowledge sharing among related stocks. The following literature review provides the necessary context to address these questions and contextualise the models evaluated in this work.

1.2 Literature Review

1.2.1 Market predictability problem

The question of whether historical, publicly available information can facilitate profitable stock trading decisions remains a central issue in financial research. The Efficient Market Hypothesis (EMH) proposed by Fama, as discussed in section 1.1, posits that asset prices in the market reflect all available information, thereby investors always trade at their true value. Nevertheless, market is not necessarily static. Under Lo's Adaptive Market Hypothesis (AMH) [3], market behaviour evolves alongside changes in participants, competitive conditions, and institutional environments. Consequently, temporary regularities may emerge, decay, or disappear across shifting market periods.

This distinction is critical for asset pricing and strategy development. A model may identify profitable relationships during one period but not retain its predictive power in subsequent periods. Financial returns are inherently noisy, their underlying statistical distributions evolve over time, and inter-asset relationships frequently shift during structural transitions or market crises. Consequently, the core objective of this study is not simply determining whether a model can fit historical returns, but whether it can suggest out-of-sample trading decisions based strictly on point-in-time information that remain profitable after accounting for turnover and transaction costs.

1.2.2 Statistical methods

Statistical time series models provide an interpretable and well-established reference baseline against more expressive machine learning approaches can be evaluated. Unlike others that learn non-linear representations from data, these methods describe temporal dependence through an explicitly specified mathematical structure.

The Autoregressive Integrated Moving Average (ARIMA) model [4] integrates autoregressive, differencing, and moving-average components. The model assumes that applying an appropriate differencing order transforms a non-stationary trend into an approximately weakly stationary series. Weak stationarity requires a constant mean and an autocovariance that depends solely on the lag between observations, rather than their absolute position in time. This assumption is essential because many financial time series exhibit stochastic trends in which the mean changes over time. Differencing mathematically removes these time-dependent components, resulting in a stable process with a constant mean and time-invariant autocovariance.

Let B denotes the backshift operator, such that $By_t = y_{t-1}$. An ARIMA(p, d, q) model represents the d-th order differenced series, $y'_t = (1 - B)^d y_t$, as

$$y'_t = c + \sum_{i=1}^{p} \phi_i y'_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t \tag{1}$$

where p is the number of autoregressive lags, d is the order of non-seasonal differencing and q is the number of lagged forecast errors. The coefficients ϕ_i capture dependence on previous observations, while θ_j capture dependence on previous innovations. The residual ε_t is assumed to be noise.

A limitation of Equation (1) is that it relies only on the history of the modelled series. ARIMAX extends the framework by incorporating K exogenous explanatory variables

$$y'_t = c + \sum_{i=1}^{p} \phi_i y'_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \sum_{k=1}^{K} \beta_k x_{k,t} + \varepsilon_t \tag{2}$$

where $x_{k,t}$ denotes an explanatory variable available at time t, and β_k represents its conditional linear contribution. In this project, the exogenous variables are engineered market technical indicators that used to forecast the subsequent stock return. Maintaining temporal ordering ensures that a forecast uses only the information available to date when trading decisions are made from the predicted return.

ARIMAX is included as one of the benchmark baselines because it combines explicit temporal dependence with the same explanatory features supplied to the machine learning forecasting baselines. Its linear and parametrically specified structure therefore provides an interpretable benchmark for assessing whether the non-linear representations learnt by machine learning models produce additional out-of-sample trading value.

1.2.3 Traditional Supervised approach

The linearity of ARIMAX maybe restrictive when relationships between market indicators and future returns are non-linear or conditional on interactions among several variables. Supervised machine learning relaxes this assumption by estimating a function

$$\hat{y}_{t+1} = f(\mathbf{x}_t) \tag{3}$$

where $\mathbf{x}_t$ contains information available at time t and $\hat{y}_{t+1}$ is the predicted subsequent return. Model parameters are learnt by reducing prediction error over labelled training observations. The model forecasts are then converted into trading positions.

The Random Forest (RF) is adopted as the baseline of supervised learning. It is an ensemble method that combines predictions from multiple decision trees [5]. Each tree is trained using a bootstrap sample of the training data, while only a randomly selected subset of predictors is considered at each split. For regression, the final prediction of it is obtained by averaging the outputs of the individual trees. Combining multiple randomisation trees generally produces a more stable predictor than relying on a single decision tree.

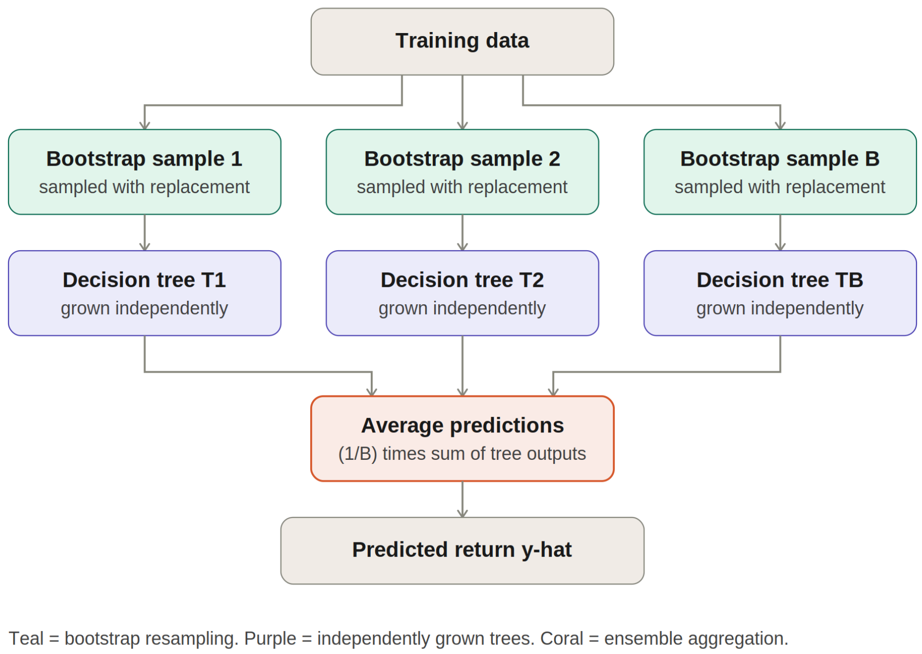

Fig. 1. Illustration of how Random Forest regressor predicts market return

However, RF has a significant limitation that it does not model chronological order intrinsically. Temporal information must be supplied through lagged observations, rolling statistics and technical indicators contained in $\mathbf{x}_t$. It also predicts using combinations of patterns represented in the training data and may adapt poorly when the relationship between predictors and returns changes outside the historical range. This baseline therefore provides a useful transition between explicit statistical time-series modelling and deep sequence architecture that learns representations directly from ordered windows of observations.

1.2.4 Deep Sequence Models

Deep learning, particularly sequence models, addresses the limitations of traditional supervised ML by learning temporal representations from ordered sequence data. Given a window of market features $\mathbf{X}_{t-L+1:t}$ of length L, these models estimate

$$\hat{y}_{t+1} = f_{\boldsymbol{\theta}}\left(\mathbf{X}_{t-L+1:t}\right) \tag{4}$$

where the parameters $\boldsymbol{\theta}$ and the internal representation of the observation window are learnt jointly by the model. This reduces reliance on a single manually selected lag structure, although the resulting flexibility also increases computational cost and the risk of overfitting noisy financial data.

The Long Short-Term Memory (LSTM) network [6] serves as a widely adopted deep sequence-processing baseline. LSTM is a recurrent neural architecture specifically designed to address the vanishing gradient problem encountered by conventional Recurrent Neural Networks. Its input, forget, and output gates regulate the flow of information into the memory state, determine the duration of information retention, and influence the model's output. This architecture enables LSTM networks to capture dependencies across various segments of the observation window. In the context of stock forecasting, LSTM models can learn how the relevance of momentum, volatility, and other indicators evolves over recent observations. However, recurrent processing remains inherently sequential and less computationally efficient than attention mechanisms. Furthermore, the memory mechanisms of LSTM do not prevent the model from learning relationships that may not generalise across different market regimes.

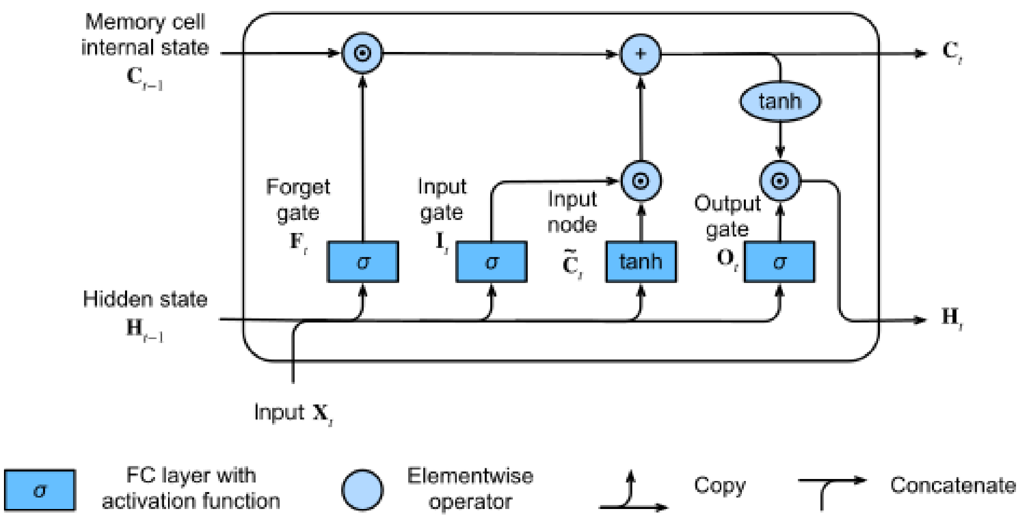

Fig. 2. Illustration of an LSTM cell [7]

The Temporal Convolutional Network (TCN) [8] provides a structurally different approach to sequence modelling. A TCN applies 1D convolutions so that the representation at time t depends only on observations available at or before t. Dilated convolutions allow the receptive field to cover increasingly distant observations without requiring a proportionate increase in network depth. Convolutional operations can also be evaluated in parallel and often provide a shorter gradient path than recurrent processing. The comparison between LSTM and TCN therefore examines whether gated recurrence or causal convolution provides a more useful representation of short market histories.

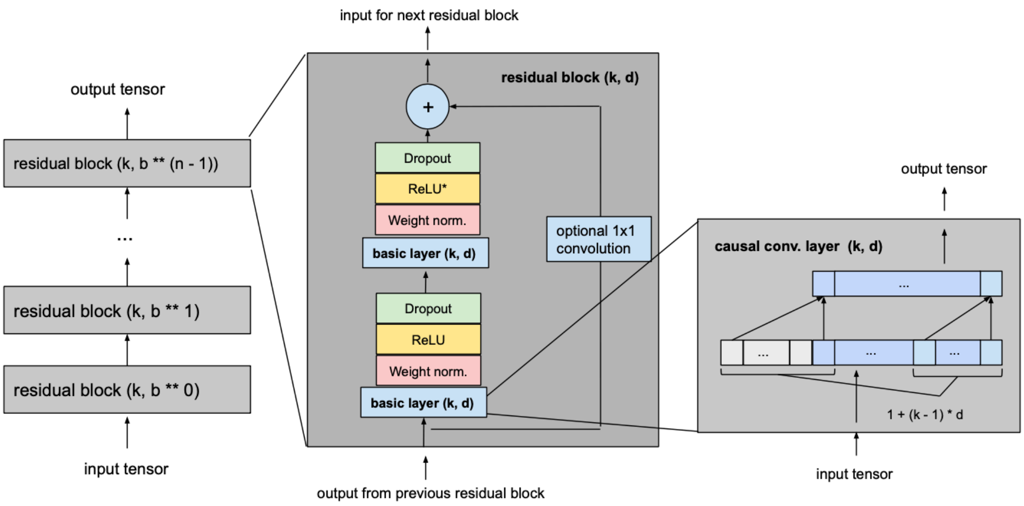

Fig. 3. The TCN architecture with a kernel size k and a dilation base b [9]

The Transformer architecture offers a third method for learning temporal representation. Unlike models that process observations recurrently or through a fixed convolutional receptive field, it uses self-attention to construct each representation as a weighted combination of observations elsewhere in the input sequence [10]. This mechanism can model interactions between distant time steps directly and allows parallel processing of the complete observation window. Because self-attention alone does not encode chronological order, positional information must be added to the input representation. In stock-return forecasting, this allows the model to distinguish, for example, an indicator observed immediately before the decision date from the same indicator observed earlier in the window. However, the flexibility of attention mechanism introduces considerable parametrisation compared with the short, noisy sequences typically available in financial data. Consequently, the Transformer used in this study is an encoder-only forecasting model whose capacity is deliberately constrained.

Through gated recurrent memory, causal dilated convolution, and attention-based representation, the LSTM, TCN, and Transformer encoder architectures offer three distinct methods for extracting temporal features. By training each model on the same chronological inputs to target next-period returns, a unified backtesting framework can reveal whether these structural variations produce profitable trading decisions or simply fit in-sample data more closely.

1.2.5 Reinforcement Learning

The preceding models treat stock trading as a forecasting problem. A prediction is first produced and then converted into a position through a separate decision function. Reinforcement Learning instead formulates trading as sequential decision making process, allowing actions and their portfolio consequences to be represented within the learning problem [11].

A Reinforcement Learning (RL) problem is commonly defined as the Markov Decision Process (MDP) consisting of states, actions, transition dynamics and rewards. At time t, an agent observes a state s_t, selects an action a_t according to a policy $\pi(a_t|s_t)$ and receives a reward r_{t+1} after the environment transitions to s_{t+1}. The objective is to maximise expected discounted return:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \right] \tag{5}$$

where the discount factor $\gamma \in [0, 1]$ determines the relative importance of future rewards.

In trading, the state may comprise a window of market indicators along with the agent's previous position. The action determines the subsequent portfolio position, while the reward quantifies the resulting portfolio return after accounting for position changes and implementation costs. Incorporating the previous position is essential because the cost of an action depends not only on the selected position but also on the turnover required to achieve it. RL can therefore integrate prediction, position selection, and trading costs more directly than traditional forecasting models [12]. Three RL algorithms evaluated in this study are Deep Q-Network (DQN), Advantage Actor Critic (A2C), and Proximal Policy Optimisation (PPO). They, respectively, represent value-based, actor-critic, and constrained policy-gradient learning methods.

DQN is a value-based deep reinforcement learning algorithm designed for discrete action spaces [13]. It approximates the optimal action-value function $Q^*(s_t, a_t)$, representing the expected discounted return from selecting action a_t in state s_t and subsequently following an optimal policy. The network is trained towards the Temporal Difference (TD) target

$$y_t^{DQN} = r_{t+1} + \gamma(1 - d_{t+1}) \max_{a'} Q\left(s_{t+1}, a'; \theta^-\right) \tag{6}$$

where r_{t+1} is the observed reward, $d_{t+1} \in \{0, 1\}$ indicates whether the transition terminates an episode, and θ^- denotes the parameters of a periodically updated target network. During training, an ϵ-greedy policy balances exploration and exploitation, while past transitions are sampled from an experience replay buffer to reduce the temporal correlation between consecutive updates. The target network is held fixed between scheduled updates, preventing the regression target from changing after every optimisation step.

A primary limitation of the standard Deep Q-Network (DQN) is that the maximisation operation relies on identical value estimates for both action selection and evaluation. This approach can lead to systematic selection of actions with overestimated values, resulting in an upward bias in the Temporal Difference (TD) target. The Double Deep Q-Network (DDQN) mitigates this issue by decoupling action selection from action evaluation [14].

$$a_{t+1}^{*} = \arg\max_{a'} Q\left(s_{t+1}, a'; \theta\right), \; y_t^{DQN} = r_{t+1} + \gamma(1 - d_{t+1}) Q\left(s_{t+1}, a_{t+1}^{*}; \theta^{-}\right). \tag{7}$$

The online network with parameters θ therefore selects the next action, whereas the target network with parameters θ^{-} evaluates it. This isolation does not eliminate the estimation error, but it reduces the maximisation bias associated with the standard DQN.

The value-based baseline implemented in this project follows the DDQN update rather than the original DQN target. It uses DDQN action evaluation with experience replay, a periodically synchronised target network, Huber loss, gradient clipping, and decaying ϵ-greedy exploration. A discrete action set of short, neutral, and long position levels makes a Q-learning method directly applicable.

Actor-critic methods instead maintain both an actor, which parametrise the policy, and a critic, which estimates the value of the current state. The critic supplies an advantage estimate indicating whether an action performed better or worse than expected. Actor-critic methods have been used in asynchronous settings (A3C), where the experience of multiple learners can diversify the updates [15]. A2C is its variant that uses advantage estimate to update the policy while simultaneously fitting the critic. This can reduce the variance of a pure policy-gradient estimator, yet large policy updates may destabilise learning.

$$L^{CLIP}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t, \mathrm{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_t\right)\right] \tag{8}$$

where $\hat{A}_t$ is the estimated advantage and ϵ limits the benefit obtainable from an excessively large policy change [16]. PPO therefore provides a more conservative alternative to the direct actor-critic update while retaining an explicit stochastic policy.

These core algorithms determine how a policy is learnt, and this study evaluates three organisational structures. In the joint structure, a single agent observes the combined market state and simultaneously selects a position for each stock selection. This enables shared representations and direct cross-stock conditioning, but creates a large state-action space and requires the policy to handle heterogeneous stock dynamics. In the independent structure, a separate agent is trained for each stock. This reduces the complexity of each individual task and improves adaptability to specific stock price patterns, yet prevents useful information from being transferred between related environments. The third structure is Group-Agent Reinforcement Learning (GARL), introduced by Wu and Zeng [17]. GARL is distinct from Multi-Agent Reinforcement Learning (MARL). In MARL, agents interact within a shared environment, and their actions may be cooperative or competitive. In contrast, under GARL, each agent learns within its own environment and is responsible for its specific task, while exchanging learning knowledge with agents engaged in related tasks. Wu and Zeng introduced the Decentralised Distributed Asynchronous Learning (DDAL) method, where agents initially learn locally and subsequently exchange knowledge asynchronously. The shared information is weighted based on the contributing agent's experience and the relevance of the source and receiving tasks. This approach aims to accelerate or enhance individual learning without consolidating agents under a single centralised policy.

This GARL-agent organisational framework has a natural interpretation in stock trading. Each stock defines a separate trading environment with its own return process, while stocks may occasionally share market-wide, sectoral or behavioural regularities. Independent agents cannot exploit these relationships, and a joint agent may apply excessive parameter sharing. GARL provides an intermediate structure in which stock-specific agents retain separate policies but exchange learning signals after an initial period of isolated training. The original GARL framework has demonstrated the potential of asynchronous knowledge sharing in RL benchmarks, but its effectiveness in noisy, non-stationary, cost-sensitive stock trading environments remains uncertain.

Knowledge sharing is not necessarily beneficial in stock trading. A gradient generated in one stock environment may conflict with the learning direction of another, particularly when relationships change across market regimes. This research project therefore also evaluates a selective GARL extension that only admits peer knowledge only when the source task has positive relevance to the receiving task and the shared gradient is positively aligned with the receiver's current local gradient. This variant is intended to reduce negative transfer while retaining the decentralised and asynchronous structure of GARL.

1.3 Research Gap and Motivation

1.4 Project Aim and Research Questions

1.5 Project Objectives

1.6 Project Scope and Contributions

1.7 Report structure

References

[1] SEC, "Introduction to investing," 2026 (cited on p. 7).

[2] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, vol. 25, pp. 383–417, 1970 (cited on p. 7).

[3] A. W. Lo, "The adaptive markets hypothesis: Market efficiency from an evolutionary perspective," *The Journal of Portfolio Management*, vol. 30, pp. 15–29, 2004 (cited on p. 8).

[4] G. Box and G. Jenkins, *Time Series Analysis: Forecasting and Control*. Holden-Day, 1970 (cited on p. 8).

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001 (cited on p. 10).

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997 (cited on p. 11).

[7] A. Zhang et al., "Dive into deep learning," in Cambridge University Press, 2023, ch. 10 (cited on p. 11).

[8] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modelling," *arXiv:1803.01271*, 2018 (cited on p. 11).

[9] F. Lässig, *Temporal convolutional networks and forecasting - unit8*, 2021. [Online]. Available: `https://unit8.com/resources/temporal-convolutional-networks-and-forecasting/` (cited on p. 12).

[10] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017, pp. 5998–6008 (cited on p. 12).

[11] R. Sutton and A. G. Barto, *Reinforcement learning: an introduction (2nd ed.)* MIT press, 2018 (cited on p. 13).

[12] J. Moody and M. Saffell, “Learning to trade via direct reinforcement,” *IEEE Transactions on Neural Networks*, vol. 12, pp. 875–889, 2001 (cited on p. 13).

[13] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015 (cited on p. 13).

[14] H. Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100 (cited on p. 14).

[15] V. Mnih et al., “Asynchronous methods for deep reinforcement learning,” *arXiv:1707.06347*, 2016 (cited on p. 14).

[16] J. Schulman et al., “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017 (cited on p. 14).

[17] K. Wu and X. J. Zeng, “Group-agent reinforcement learning,” *International Conference on Artificial Neural Networks*, pp. 37–48, 2023 (cited on p. 15).

Appendices

A Project outline

Project outline as submitted at the start of the project is a required appendix. Put here.

B Risk assessment

Risk assessment is a required appendix. Put here.